UNIVERSITY OF BUEA
Faculty of Engineering and Technology
Department of Computer Engineering and Technology

CEF488: Systems and Network Programming
Laboratory Report

Lab 2: Named Pipes, Advanced Multiplexing & Real-Time Signals

Centralized Logging System Using FIFOs and epoll

Member Name	**Matricule Number**	Date
Emengu Njualem Fondong	FE25A673	07/05/2026
Moukam Hako	FE223A096	07/05/2026
Nyonyoh divine-Fortune Banfila	FE23A128	07/05/2026
Tabot Ghislain Orock	FE23A146	07/05/2026

Course instructor:

Mr. FORCHA GLEN

ACADEMIC YEAR 2025/2026

2. HONESTY REPORT

The table below indicates the contribution of each team member to each aspect of the laboratory exercise.

Task	Responsible Member(s)	Matricule
Task 1: Log Server (logserver.c)	Tabot Ghislain Orock	FE23A146
Task 2: Client Programs (logclient.c)	Nyonyoh divine-Fortune Banfila	FE23A128
Task 3: Dynamic FIFO Addition (controlclient.c) and Writing of PowerPoint	Emengu Njualem Fondong	FE25A062
Task 4,Report Writing, answering of prelab	Moukam Hako Bellamy Steve	FE23A096

Table of Contents

3. ABSTRACT/ SUMMARY

This lab focused on building a centralized logging system using named pipes (FIFOs), epoll, and POSIX signals in Linux. The aim was to allow multiple clients to send log messages to a single server at the same time without blocking. The server program creates multiple FIFOs and monitors them using epoll in edge-triggered mode. Messages are sent in the format “LEVEL|message and CRITICAL messages” are forwarded to a separate alert FIFO. A control FIFO was also implemented to allow new FIFOs to be added while the server is running.

During implementation, several issues were encountered, especially with edge-triggered epoll and FIFO blocking behaviour. These problems were solved by using non-blocking I/O and reading until EAGAIN. The system was able to handle multiple clients correctly and dynamically add new FIFOs without restarting the server.

4. OBJECTIVE

By the end of this laboratory, the student should be able to:

- Create and manipulate named pipes (FIFOs) using mkfifo(), open(), read(), and write() system calls, and understand how FIFOs differ from anonymous pipes.
- Configure the Linux epoll interface in edge-triggered mode to efficiently monitor multiple file descriptors simultaneously without polling.
- Guarantee atomic message delivery by keeping individual writes within the PIPE_BUF boundary, preventing interleaving across concurrent writers.
- Implement a dynamic file-descriptor management scheme that allows new FIFOs to be added to an existing epoll set at runtime through a control channel, without restarting the server.
- Handle asynchronous process-termination events by detecting EOF conditions (read() returning zero) and re-opening FIFOs so the server remains ready for new clients.
- Apply proper signal handling using sigaction() for graceful shutdown, and understand the interaction between signals and blocking system calls.

5. PRELAB ANSWERS

Question 1: Unnamed Pipes vs. Named Pipes

An unnamed pipe is created with the pipe() system call and exists only in kernel memory; it has no filesystem path, so it can only be shared between a parent and its descendants through inherited file descriptors. A named pipe (FIFO) is created with mkfifo() and appears as a special file in the filesystem, meaning any process with appropriate permissions can open it by path regardless of its relationship to the creator.

A suitable scenario for named pipes: a web server process needs to stream access logs to a separate log-aggregation daemon. Because these are unrelated processes (neither forked the other), they cannot share an anonymous pipe. A FIFO at /var/run/webserver.log allows the web server to write and the aggregator to read without any parent-child relationship or shared memory region.

Question 2: PIPE_BUF

PIPE_BUF is the kernel-defined threshold (4096 bytes on Linux) below which a single write() to a pipe or FIFO is guaranteed to be atomic. Atomic here means the kernel will not interleave the bytes of that write with bytes from any concurrent write by another process. For a structured logging protocol, this guarantee is critical: if each log message is smaller than PIPE_BUF, a reader always receives a complete, unmangled record. If a message exceeds PIPE_BUF, the kernel may split it across multiple internal writes, corrupting the record boundary and causing the receiver to receive garbled or merged messages.

Question 3: Level-Triggered vs. Edge-Triggered epoll

In level-triggered (LT) mode, epoll_wait() reports a file descriptor as ready whenever data remains in its buffer, even across multiple calls. In edge-triggered (ET) mode, epoll_wait() fires exactly once when the state transitions from no-data to some-data. After an ET notification the application must drain the descriptor completely (reading until EAGAIN) before the next edge can be detected; failure to do so means new data that arrives before the buffer empties will not trigger another notification, causing messages to be silently dropped.

ET mode is preferred in high-throughput servers because it reduces the number of wake-ups from epoll_wait(), avoiding redundant system calls when data is being processed. LT mode is simpler to program and is preferable in cases where the application can handle partial reads across events without risk of missing data.

In this lab, edge-triggered mode was chosen for efficiency and pedagogical alignment with the lab specification.

Question 4: Creating a Named Pipe

A named pipe is created with the mkfifo() system call:

```
int mkfifo(const char *pathname, mode_t mode);
```

Typical permissions are 0666 (read/write for owner, group, and others), which after applying the process umask commonly yields 0644. The FIFO is then opened with open() using O_RDONLY | O_NONBLOCK for the server reader, or O_WRONLY for a client writer.

Question 5: Adding a New FIFO to an Existing epoll Set

The procedure is as follows. First, the target FIFO must already exist in the filesystem (created with mkfifo() or verified with stat()). Second, the server opens it with open(path, O_RDONLY | O_NONBLOCK) to obtain a file descriptor. Third, an epoll_event structure is populated with events = EPOLLIN | EPOLLET and data.fd = fd. Fourth, epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev) registers the descriptor with the existing epoll instance. The server's internal registry (FifoEntry table in this implementation) must also be updated so that subsequent events can be correlated with the correct FIFO path and partial-message buffer.

Question 6: Real-Time Signals vs. Standard Signals

Standard signals (SIGINT, SIGTERM, etc.) are not queued: if the same signal is delivered multiple times before the process handles it, only one delivery is guaranteed. Real-time signals (SIGRTMIN through SIGRTMAX) are fully queued: every sent signal is delivered, and they carry a user-defined integer or pointer payload through sigqueue().

The sigqueue() function allows a sender to attach a si_value union to the signal, enabling the receiver to identify which resource (e.g., which FIFO) triggered the notification without additional bookkeeping. This is useful in designs where each monitored resource is associated with a distinct real-time signal number.

Question 7: Writes Larger Than PIPE_BUF

If a client calls write() with a message larger than PIPE_BUF, the kernel may split the write into multiple internal operations, allowing bytes from other concurrent writers to be interleaved in between. The reader will then receive a corrupted message that merges fragments from two different clients. To guarantee atomic delivery, messages must be kept strictly below PIPE_BUF. If a message is inherently large, the sender must implement an application-level framing protocol (e.g., a length-prefix header followed by chunks each smaller than PIPE_BUF) and the receiver must reassemble the chunks before processing.

Question 8: Handling EAGAIN in Non-Blocking Mode

When a FIFO is opened with O_NONBLOCK, read() returns immediately rather than blocking when no data is available. In this state it returns -1 and sets errno to EAGAIN (or equivalently EWOULDBLOCK). In edge-triggered epoll mode, the application is required to read until it sees EAGAIN to ensure all buffered data has been consumed before the next edge notification. Without this loop, data present in the kernel buffer would not trigger another EPOLLIN event, and messages

would remain unread indefinitely until new data caused a fresh edge. EAGAIN is not an error condition; it is the canonical signal that the pipe is temporarily empty and the read loop should exit.

6. METHODOLOGY

6.1 Design Approach

The system was designed to support multiple clients sending log messages to one server process. The server uses epoll to monitor several FIFOs at the same time.

All FIFOs were opened using O_NONBLOCK so that the server would not freeze while waiting for input. epoll was configured using EPOLLIN and EPOLLET. Since edge-triggered mode only sends notifications once when new data arrives, the server had to keep reading until EAGAIN to avoid missing messages.

Messages were written using the format:

Level|message

Where each message was kept smaller than PIPE_BUF so that writes remain atomic. This prevents two clients from mixing their messages together inside the FIFO.

A control FIFO called /tmp/control_client was added so that new FIFOs could be registered while the server is already running. The server reads commands like:

ADD /tmp/fifoname

and then adds the new FIFO into the epoll instance dynamically.

Another issue was handling EOF when clients disconnected. When this happened, the server received EPOLLHUP. Instead of deleting the FIFO, the server reopened it so that future clients could still connect.

6.2 Source File Structure

Source File	Responsibility
logserver.c	Main server program. Creates FIFOs, handles epoll events, processes messages, and forwards alerts.
logclient.c	Sends log messages into a FIFO using a single write().
controlclient.c	Sends ADD commands to the control FIFO for dynamic FIFO registration.
Makefile	Compiles all programs and provides clean targets.

6.3 Key Functions in logserver.c

Function	Purpose
main()	This starts the server and runs the event loop.
create_and_open_fifo()	Creates the FIFO if absent (mkfifo), opens it non-blocking, returns the fd.

Function	Purpose
add_fifo_to_epoll()	This Adds a file descriptor into epoll monitoring.
drain_fifo()	This Reads data continuously until EAGAIN.
process_message()	This Parses LEVEL and message content.
handle_control_command()	This Handles ADD commands from the control FIFO.
cleanup()	Closes descriptors and removes FIFOs during shutdown.
timestamp()	Returns a static HH:MM:SS string using localtime() and strftime().

6.4 Libraries and Tools

- POSIX APIs: mkfifo(3), open(2), read(2), write(2), epoll_create1(2), epoll_ctl(2), epoll_wait(2), sigaction(2), alarm(2).
- Standard C library: stdio.h, stdlib.h, string.h, time.h.
- Build tools: GCC with -Wall -Wextra, GNU Make.
- Development environment: VirtualBox running Ubuntu 24.04 LTS, terminal-based testing with multiple bash sessions.

7. RESULTS

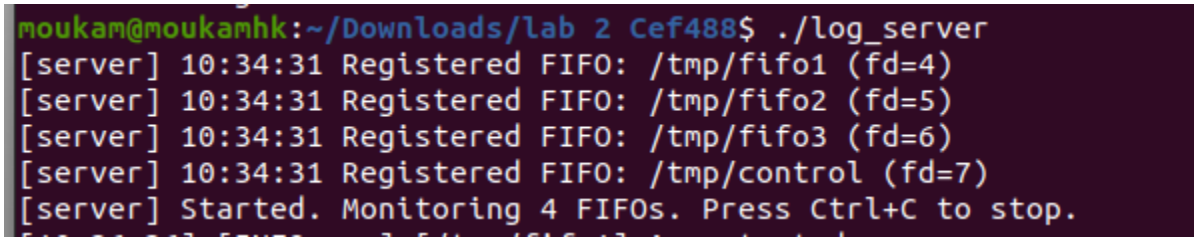
7.1 Compilation

All three source files compile without warnings under GCC with the -Wall -Wextra flags:

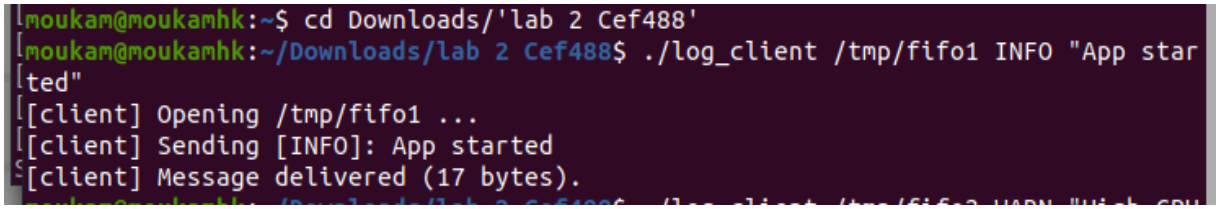
```
$ make all
gcc -Wall -Wextra -o log_server log_server.c
gcc -Wall -Wextra -o log_client log_client.c
gcc -Wall -Wextra -o control_client control_client.c
```

7.2 Task 1: Basic Multi-FIFO Logging (Terminal Output)

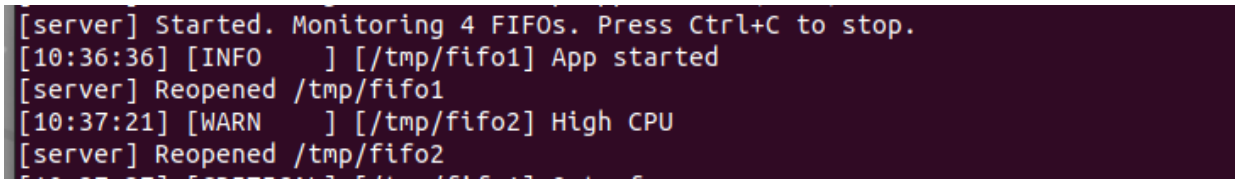
The server was launched in Terminal 1. It created and registered all FIFOs:



Log clients were launched from separate terminals (Terminal 2 and Terminal 3):

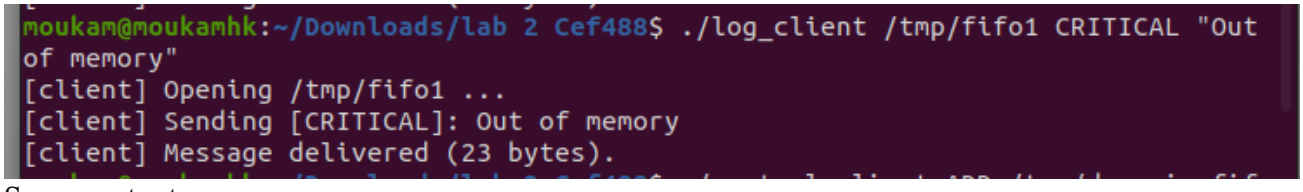


Corresponding server output:

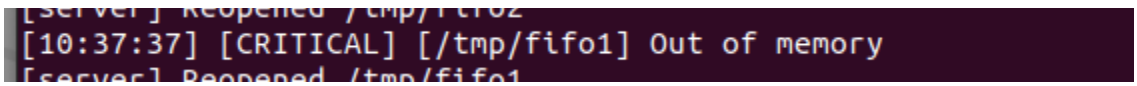


7.3 Task 2: Critical Message Alert Forwarding

A CRITICAL message was sent from Terminal 3. An alert reader (cat /tmp/alert_fifo) was active in Terminal 4:



Server output:



Alert FIFO output (Terminal 4):

```
ALERT [14:23:14] from [/tmp/fifo2] Out of memory!
```

```
[server] Reopened /tmp/fifo2
[10:37:37] [CRITICAL] [/tmp/fifo1] Out of memory
```

Critical messages were correctly forwarded to /tmp/alert_fifo atomically with a single write().

7.4 Task 3: Dynamic FIFO Registration

A custom FIFO was registered at runtime without restarting the server:

```
moukam@moukamhk:~/Downloads/lab 2 Cef488$ ./control_client ADD /tmp/dynamic_fifo
[control_client] FIFO /tmp/dynamic_fifo not found; creating it...
[control_client] Created FIFO: /tmp/dynamic_fifo
[control_client] Connecting to control FIFO /tmp/control ...
[control_client] Sent: ADD /tmp/dynamic_fifo
[control_client] Server should now monitor: /tmp/dynamic_fifo
```

Server output:

```
[server] 10:38:03 Control command: 'ADD /tmp/dynamic_fifo'
[server] 10:38:03 Registered FIFO: /tmp/dynamic_fifo (fd=9)
```

Log messages sent to /tmp/my_app_log were immediately received by the server.

7.5 EOF and FIFO Reopen

When a client process terminated (closing its write end), the server detected EPOLLHUP, closed the stale fd, and reopened the same FIFO path non-blocking:

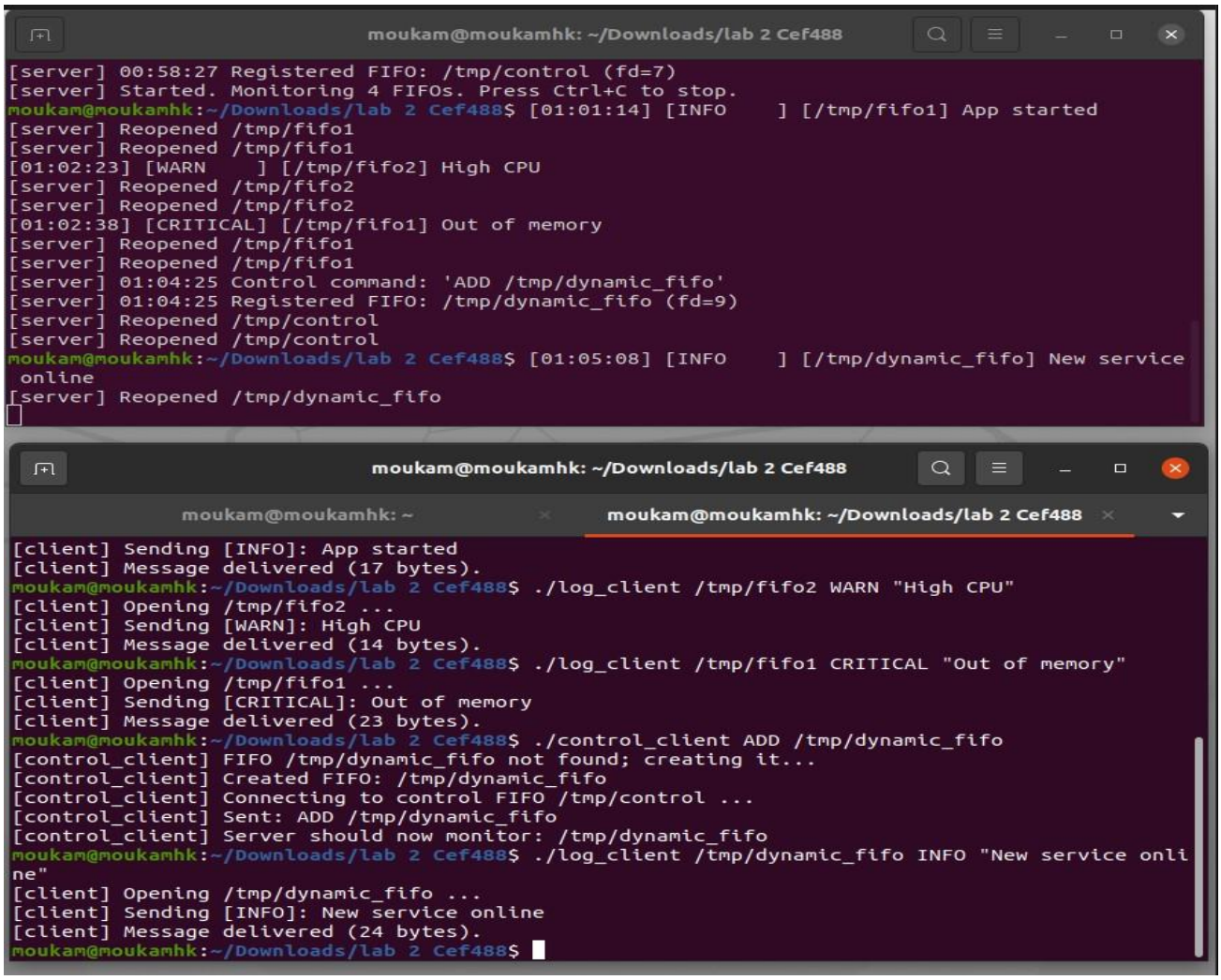
```
[server] Reopened /tmp/control
[10:38:24] [INFO    ] [/tmp/dynamic_fifo] New service online
[server] Reopened /tmp/dynamic_fifo
```

Subsequent clients could connect to /tmp/fifo1 without any server restart.

7.6 Source File Summary

File	Lines of Code	Role
logserver.c	480	Central logging server with epoll, alert forwarding, control channel, and EOF recovery.
logclient.c	90	Log message sender with atomic write and 5-second alarm timeout.
controlclient.c	100	Dynamic FIFO registration client; auto-creates FIFO if absent.

File	Lines of Code	Role
Makefile	10	Build targets: all, server, client, control, clean.



OVERALL VIEW OF THE THE DIFFERENT COMMANDS THAT WHERE PERFORMED

8. ANALYSIS AND DISCUSSION

8.1 How the system works

The implementation satisfies all mandatory tasks (Tasks 1, 2, and 3) and addresses the additional considerations specified in the lab handout.

The implementation completed all the required tasks from the lab. The server successfully monitored multiple FIFOs at the same time and processed incoming messages correctly.

CRITICAL messages were forwarded to the alert FIFO as expected. The control FIFO also worked correctly and allowed new FIFOs to be added without restarting the server.

One important observation during testing was that edge-triggered epoll can easily miss data if the FIFO is not completely drained. At first, some messages stopped appearing because the server only read once per event. This was fixed by continuously reading until EAGAIN.

8.2 Problems Encountered

- FIFO Blocking:

Initially, opening the alert FIFO using O_WRONLY caused errors when no reader process was attached. The server returned ENXIO. This was fixed by opening the FIFO with O_RDWR instead.

- EPOLLHUP After Client Exit:

When a client terminated, the server received EPOLLHUP. At first the FIFO stopped accepting new writers. The solution was to close and reopen the FIFO immediately instead of deleting it.

- Partial Reads:

Sometimes multiple messages arrived in one read operation, while other times a message arrived partially. To solve this, partial data was stored in a buffer until a newline character was detected.

- Control FIFO Waiting Forever:

If the server was not running, control_client blocked while trying to open /tmp/control. A 5-second alarm timeout was added to prevent the client from hanging forever.

9. CONCLUSION

This lab showed how FIFOs and epoll can be combined to build a simple centralized logging system in Linux. The system was able to monitor several FIFOs simultaneously and process messages from multiple clients correctly. The most important lesson from the lab was understanding how edge-triggered epoll works. If data is not fully drained until EAGAIN, the server may stop receiving future notifications. Another important concept was PIPE_BUF atomicity, which prevents concurrent clients from mixing messages together.

Overall, the implementation achieved all the required objectives and provided practical experience with Linux system programming, non-blocking I/O, and inter-process communication.

10. REFERENCES

1. Kerrisk, M. (2010). The Linux Programming Interface. No Starch Press. (Chapters 44 to 46: Pipes and FIFOs; Chapter 63: I/O Multiplexing).
2. Linux man-pages project. mkfifo(3), pipe(7), epoll(7), epoll_create1(2), epoll_ctl(2), epoll_wait(2), read(2), write(2), sigaction(2), fcntl(2). Available at: https://man7.org/linux/man-pages/
3. IEEE Std 1003.1-2017 (POSIX.1-2017). Pipes and FIFOs; Atomic writes. The Open Group.
4. Stevens, W. R., & Rago, S. A. (2013). Advanced Programming in the UNIX Environment (3rd ed.). Addison-Wesley. Chapter 15: Interprocess Communication.
5. Love, R. (2013). Linux System Programming (2nd ed.). O'Reilly Media. Chapter 4: Advanced File I/O (epoll).

11. APPENDIX

A. Makefile

```
CC      = gcc
CFLAGS  = -Wall -Wextra -g
TARGETS = log_server log_client control_client

all: $(TARGETS)

log_server: log_server.c
    $(CC) $(CFLAGS) -o $@ $<

log_client: log_client.c
    $(CC) $(CFLAGS) -o $@ $<

control_client: control_client.c
    $(CC) $(CFLAGS) -o $@ $<

clean:
    rm -f $(TARGETS)
    rm -f /tmp/fifo1 /tmp/fifo2 /tmp/fifo3 /tmp/alert_fifo /tmp/control
```

B. Full Source Code log_server.c

```
static void drain_fifo(FifoEntry *entry) {
    char buf[BUF_SIZE];
    while (1) {
        ssize_t n = read(entry->fd, buf, sizeof(buf));
        if (n < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) break;
            perror("read fifo"); break;
        }
        if (n == 0) break; /* EOF */
        /* accumulate in partial[], split on '\n' */
        ...
    }
}
```

C. Full Source Code log_client.c

```
int len = snprintf(msg, sizeof(msg), "%s|%s\n", level, text);
if (len >= PIPE_BUF) { /* reject : too large */ }
ssize_t written = write(fd, msg, len); /* atomic */
```

D. Full Source Code of control_client.c

```
if (mkfifo(argument, 0666) < 0) { perror("mkfifo"); }
int fd = open(CONTROL_FIFO, O_WRONLY);
write(fd, msg, len); /* atomic: "ADD /path\n" */
```